

Build: Multi-Agent Smart Elevator Fleet Coordinator

Goal

Build a polished, fully working, well-tested **multi-agent simulation + live web dashboard** in which autonomous agents coordinate a fleet of elevators in a multi-storey building. This is a university case study (course: *Fundamentals of AI*, textbook: Russell & Norvig, AIMA 4e; team of 4). Every design decision must visibly map to course concepts: agent types, PEAS, environment properties, state-space search (BFS, UCS, Greedy, A*), local search (hill climbing, simulated annealing), adversarial search (minimax, alpha-beta), rule-based reasoning (forward chaining).

Optimise for this grading rubric:

Review 1	Review 2
PEAS formulation (3)	Tool/package selection & setup (3)
Environment & agent analysis (3)	Multi-agent execution & interaction (3)
Algorithmic modeling & search strategy (3)	Demo quality & testing scenarios (3)
Q&A & presentation (1)	Code structure & scalability (1)

Tech stack (pin versions; justify each in docs)

- **Python 3.12 via uv** (demo laptop is Ubuntu 22.04 with system Python 3.10 — **uv sync** must fetch 3.12 itself).
- **Mesa 3.x** (`mesa>=3.3, <4`) for Agent/Model, seeded RNG and DataCollector. Use the Mesa 3 API only: `Agent.__init__(self, model, ...)` + `super().__init__(model)`, no schedulers (staged activation via `model.agents_by_type[Cls].do("stage")`), `agent.remove()`, randomness only from `model.random`. Check the installed version's docs; avoid deprecated APIs.
- **FastAPI + uvicorn** (REST + WebSocket streaming), pydantic config models, PyYAML, Typer CLI.
- **Frontend**: single-page vanilla HTML/CSS/JS + Canvas, no build step; any JS library (e.g. Chart.js) vendored in `static/vendor/` so the demo works offline.
- pandas + matplotlib (benchmark tables/PNGs), pytest (+ coverage), ruff.

Environment model

- Building of N floors (0 = lobby) and M cars, all from YAML (defaults: 15 floors, 4 cars, capacity 10, 2 s/floor, 2 s door open, 2 s door close, 1 s per boarding passenger). 1 tick = 1 simulated second; frontend interpolates for smooth animation.
- Passengers arrive by a Poisson process with a time-varying rate and an origin–destination profile per traffic pattern (up-peak, down-peak, lunch two-way, inter-floor, light).
- Floors have up/down hall buttons and an occupancy sensor (waiting count). A passenger's destination is unknown until they board and press a car button → agents must **replan closed-loop**.
- Environment classification (show with justification in UI + docs): partially observable, multi-agent (cooperative), stochastic, sequential, dynamic, discrete (floors/ticks, discretised from continuous), known physics / unknown demand (learned online).

Agents (each a Mesa Agent; agent type + PEAS in docstring and in the UI)

1. **PassengerAgent** — simple reflex: arrive, press hall button, board a car serving its direction if there is space, press car button, alight; records timestamps.
2. **FloorAgent** (one per floor) — model-based reflex: tracks hall-call state, sends REQUEST to the dispatcher on a new call; fairness/aging: re-escalates with higher urgency if unserved > 60 s (no starvation).
3. **ElevatorAgent** (one per car) — goal + utility-based: internal model (position, direction, door state, load, assigned pickups, car calls); plans its stop sequence with A*; bids in auctions with marginal cost; executes move/door actions; reports INFORM/FAILURE.
4. **DispatcherAgent** — coordinator/auctioneer: runs Contract Net per hall call, periodic global reassignment by simulated annealing, re-auctions calls of failed cars.
5. **TrafficMonitorAgent** — learning agent: online EWMA estimates of per-floor arrival rates + rule-based traffic-pattern classification; tells the dispatcher to adapt cost weights and parking policy.
6. **SafetyAgent** — knowledge-based: forward-chaining production rules (fire alarm → recall all cars to lobby, doors open, block hall calls; overload → hold doors, refuse boarding; car fault → out of service + reassign). Fired rules shown in the UI. Bonus: same rules as Prolog in [docs/safety_rules.pl](#).

System-level PEAS to refine per agent — P: avg/95th-pct wait, ride time, % waits > 60 s, energy, safety, fairness; E: building, shafts, floors, passengers, other cars; A: motor up/down/stop, doors, hall lanterns/displays, messages; S: position encoder, load & door sensors, car/hall buttons, occupancy sensors, messages.

Agent communication (make interaction visible)

- FIPA-ACL-style `Message(performative, sender, receiver, conversation_id, content, tick)` on a MessageBus; performatives REQUEST, CFP, PROPOSE, REFUSE, ACCEPT_PROPOSAL, REJECT_PROPOSAL, INFORM, CANCEL, FAILURE. Every message is logged.
- **Contract Net per hall call:** Floor REQUEST → Dispatcher CFP to all cars → cars PROPOSE (cost + breakdown) or REFUSE (full / out of service / fire mode) → ACCEPT lowest, REJECT others → winner INFORMs ETA to floor (hall lantern shows assigned car).
- Deterministic tick stages: sense → communicate/bid → decide → act → collect metrics.

Algorithms (the core of grading — clean, generic, AIMA-style)

- Generic `SearchProblem` (initial_state, actions, result, action_cost, is_goal, h) with **BFS, UCS, Greedy best-first, A*** sharing one node/frontier implementation; each returns path, cost, nodes expanded, max frontier size, runtime.
- **Car routing as state-space search:** state = (current floor, frozenset of pending stops: pickups (`floor, dir, weight=waiting count`) and drop-offs (`floor, weight=riders`)); action = go to next pending stop (same-floor stops merged); legality (collective control): an empty car may take any pickup; with riders on board, pickup (`f, UP`) only if $cur \leq f \leq$ every pending drop-off (mirror for DOWN), so riders are never carried backwards; step cost = (travel + dwell time) × total weight of still-unserved stops + energy term (floors travelled, stops), so path cost = $\sum \text{weight} \times \text{arrival time} + \text{energy}$; heuristic $h = \sum w_i \cdot \text{travel}(cur, f_i) + \text{energy lower bound}(\text{line span})$ — admissible and consistent (prove in docs, verify in tests). Above 10 pending stops fall back to LOOK ordering (bounded rationality; document it).
- **Bid** = marginal cost of inserting the call into the car's plan, split into wait, ride, crowding and energy with weights W1–W4 chosen by the current traffic pattern.
- **Global reassignment every 15 s:** simulated annealing (plus a hill-climbing variant) over the assignment of not-yet-picked-up calls (neighbours: move/swap one call), cheap insertion estimate as objective, hysteresis threshold to avoid oscillation; changes go out as CANCEL/ACCEPT messages.
- **Idle-car parking:** (a) lobby baseline; (b) hill climbing with random restarts minimising expected response $\sum \lambda_f \cdot \min_c |p_c - f|$; (c) minimax with alpha-beta (dispatcher = MAX picks a parking configuration, "nature" = MIN picks the next call floor among likely floors) for a worst-case guarantee; report nodes with vs without pruning.
- **Pluggable dispatch strategies** (registry) for comparison: NearestCar (reflex baseline), Collective/LOOK, CNP + A*, and Full (CNP + A* + SA + smart parking).

Web dashboard (demo quality matters most — make it look professional)

1. **Live Simulation:** animated building (shafts, smoothly moving cars coloured by state, doors, load bar, waiting passengers per floor, hall arrows coloured by assigned car). Controls: play/pause/step/reset, speed 1–50×, seed, scenario, strategy, floors/cars. Click a floor to add a passenger; buttons to inject car fault, fire alarm, passenger rush. Side panels: live KPIs; **auction panel** (last CNP round as bid bar chart per car, winner and cost breakdown); **agent inspector** (click a car → agent type, PEAS, internal state, planned route); **message log** (colour-coded performatives, filter by agent/conversation); rules-fired log; AWT-over-time chart.
2. **Search Lab:** snapshot a car's live routing problem (or a random one) and run BFS/UCS/Greedy/A* side by side (cost, nodes expanded, time, route); SA convergence curve; minimax vs alpha-beta node counts.
3. **Benchmark:** run strategies × scenarios × seeds headless; mean ± std table and charts.
4. **Theory:** PEAS table per agent, environment classification table, architecture diagram, CNP sequence diagram.

Scenarios (YAML; expected behaviour documented and asserted in tests)

`morning_up_peak`, `evening_down_peak`, `lunch_two_way`, `interfloor_light`, `car_breakdown` (car 2 fails at $t=120$ with riders → stops at next floor, riders re-queue, its calls re-auctioned, recovers at $t=300$), `fire_emergency` (recall at $t=150$), `priority_passenger` (wheelchair/VIP weight ×3), `stress_scale` (40 floors, 8 cars, heavy traffic), `demo_story` (up-peak → breakdown → fire in ~5 min, for the live presentation).

Metrics

Average wait, ride and system time; P95 and max wait; % waits > 60 s; throughput; energy (floors travelled + stops + reversals); messages per call; planner nodes expanded; compute ms/tick. Collected with Mesa DataCollector, exportable to CSV.

Testing (pytest; all must pass; ≥ 85 % coverage of core logic)

- A* cost == UCS cost on 200 seeded random routing instances; A* expands ≤ UCS; heuristic admissible and consistent; plans obey collective legality.
- Invariants over full runs: capacity never exceeded, doors never open while moving, cars within bounds, every passenger delivered after drain (no starvation), same seed ⇒ identical results.

- CNP: exactly one ACCEPT per call; out-of-service cars REFUSE; a failure re-auctions all its calls; fire mode recalls every car to the lobby and blocks hall calls.
- SA never worse than its initial assignment; alpha-beta value == minimax value with fewer nodes.
- `stress_scale` 1 simulated hour headless in reasonable time (target < 2 min; cache plans, replan only on events). API + WebSocket smoke tests.

Code structure & scalability

`src/elevator_mas/{config, model, agents/, comms/, planning/, optimization/, strategies/, traffic/, rules/, metrics, sim/, api/, web/}, configs/scenarios/*.yaml, tests/, docs/, reports/` (generated). Type hints and docstrings throughout, no global state, strategy/registry pattern, headless engine fully separate from UI, floors/cars/strategies/scenarios changeable without code edits.

Deliverables

- `README.md`: one-command setup (`uv sync`), run (`uv run elevator serve → http://localhost:8000`), headless run (`uv run elevator run --scenario ...`), benchmark (`uv run elevator bench`), tests.
- `AGENTS.md`: repo conventions and how to run/test.
- `docs/DESIGN.md`: PEAS per agent, environment analysis, agent-type justification, state-space formulation, algorithms with complexity and heuristic proofs, CNP sequence diagram (Mermaid), architecture diagram, tool-selection table (tool / why / alternatives rejected — e.g. SPADE needs an XMPP server, pygame has no dashboard, Streamlit animates poorly), work split for 4 team members.
- `docs/TESTING.md` (scenario results table), `docs/DEMO_SCRIPT.md` (5-minute live demo walkthrough), `docs/VIVA_QA.md` (20 likely examiner questions with crisp answers).
- `reports/`: benchmark CSV + PNG charts ready for slides.

How to work

- First write a short `docs/PLAN.md` with milestones, then implement end to end without stopping for confirmation.
- Milestones: **M1** engine + agents + CNP + A* + tests → **M2** live dashboard → **M3** SA, parking, safety rules, faults, all scenarios → **M4** Search Lab, benchmark, docs. Keep the app runnable after every milestone; run `ruff` and `pytest` after each.
- No placeholders, TODOs or mocked logic; everything real and reproducible by seed.
- **Done when**: all tests pass; `uv run elevator serve` shows all 4 tabs working; every scenario runs; the benchmark shows the Full strategy beating NearestCar on average wait and % long waits in peak scenarios (tune if not, and report results honestly); docs are complete. Finish with a summary of how to run it and what was built.